



**Università
di Genova**

UNIVERSITY OF GENOA
Master's Degree in Robotics
Engineering (Class LM-32)
Program Code: 11963
Academic Year: 2025/2026



Planetary Rover - Heterogeneous Multi-Robot Exploration

Artificial Intelligence for Robotics II (AI4RO2)



Student: HUSSEIN MOHAMED ABDELAZIM MOHAMED MOSTAFA ELNAGGAR

Student ID: 8786438

Email: S8786438@studenti.unige.it || eng.h.elnagar@gmail.com

June 18, 2026

1. Domain and Planning Problem Overview

The project models a multi-agent planetary exploration scenario requiring coordination between heterogeneous rovers (wheeled and legged) to collect and deliver samples to a base station. The environment features distinct terrain topologies (flat, rough, highway) restricting mobility based on hardware capabilities. The project spans two phases:

1. **Phase 1 (Classical STRIPS)**: Utilizes classical planning to validate logical coordination and task allocation.
2. **Phase 2 (PDDL+)**: Elevates the model to a Hybrid Dynamical System, introducing continuous time ($\#t$), numeric fluents, and autonomous environmental dynamics.

To avoid redundant action definitions, robot capabilities are explicitly encoded using the **can-traverse** predicate, allowing multi-agent cooperation to emerge organically from spatial constraints rather than explicit synchronization actions. In the PDDL+ extension, durative actions were explicitly avoided as they merely schedule discrete jumps without modeling continuous intermediate states. Instead, navigation was decoupled into instantaneous start/end actions modulated by continuous **Processes** and **Events**:

- **Processes**: **Maps-process** computes continuous linear distance reduction. Concurrently, **accumulate-risk** simulates terrain hazards by increasing the **risk-level** proportionally to the terrain's **risk-rate** over time. The **recover-risk** process allows active risk depletion when a rover executes a **start-rest** action.
- **Events**: To strictly enforce hardware limitations, the **risk-failure** event acts as an autonomous hard constraint. If the numeric **risk-level** exceeds **max-risk**, the event triggers immediately, rendering the rover **is-broken** and halting its velocity, effectively pruning invalid trajectories during the heuristic search.

2. Part 1: Classical Planning (STRIPS)

2.1 Domain Overview and Modeling Strategy

The proposed PDDL domain models a heterogeneous multi-robot exploration scenario utilizing Classical STRIPS planning. The environment features varying topological constraints characterized by distinct terrain types (e.g., flat, rough, mixed). To avoid redundant action definitions (e.g., creating separate navigation actions for different rovers), the model explicitly encodes robot capabilities using the **can-traverse** predicate. This ensures that the **Maps** action dynamically evaluates whether a specific rover type (wheeled or legged) possesses the mechanical capability to traverse the targeted terrain. Consequently, multi-agent cooperation emerges organically from spatial and terrain constraints rather than relying on explicitly forced coordination actions.

2.1.1 Types & Predicates (State Variables & Constraints)

This table below defines the objects in the environment and their boolean states (True/False).

Component Type	Element Name & Parameters	Description & Purpose
Type	rover, location, sample, terrain	Defines the hierarchical object classes used in the domain to ensure type safety.
Predicate	(at ?r ?l)	Tracks the current spatial location ?l of a specific rover ?r.
Predicate	(connected ?l1 ?l2)	Defines the graph topology (navigable edges) between two locations.
Predicate	(terrain-type ?l ?t)	Assigns a specific terrain type ?t (e.g., flat, rough) to a location node ?l.
Predicate	(can-traverse ?r ?t)	Crucial for Heterogeneity: Defines hardware capabilities. Maps which rover can drive on which terrain, avoiding duplicate action definitions.
Predicate	(sample-at ?s ?l)	Tracks the physical location of a sample when it is on the ground.
Predicate	(carrying ?r ?s)	Indicates that rover ?r is currently holding sample ?s in its gripper.
Predicate	(empty ?r)	Indicates the rover's gripper is free. Required to pick up a new sample.
Predicate	(delivered ?s)	The Goal State . Indicates the sample has been successfully secured at the base.
Predicate	(is-base ?l)	Flags a specific location node as the designated drop-off point for the mission.

2.1.2 Actions (Transitions & Logic)

This table below defines the operations rovers can perform, detailing what must be true before execution (Preconditions) and what changes afterward (Effects).

Action Name	Preconditions (What must be true)	Effects (State changes)	Purpose & Discussion Context
Maps	- Rover is at ?from - ?from and ?to are connected - Target has terrain-type - Rover can-traverse target terrain	- (not (at ?r ?from)) - (at ?r ?to)	Moves the rover. Note: Uses capability-based constraints instead of hardcoding separate wheeled/legged actions.
collect-sample	- Rover is at sample location - Sample is sample-at location - Rover gripper is empty	- (not (sample-at)) ground - (not (empty)) gripper - (carrying) sample	Allows the rover to pick up a target item. Enforces payload capacity (1 item at a time).
drop-sample	- Rover is at location - Rover is carrying sample	- (not (carrying)) - (empty) gripper - (sample-at) ground	Key for Cooperation: Allows a rover to leave a payload at an intermediate zone for another rover to pick up (Relay Handover).
deliver-sample	- Rover is at location - Location is-base - Rover is carrying sample	- (not (carrying)) - (empty) gripper - (delivered) sample	Fulfills the mission objective. Strictly requires the rover to be at the designated base station.

2.2 Problem 1: Single Robot, Simple Environment

Description: A fundamental sequential planning test where a single wheeled rover must retrieve a sample from a flat terrain location and return it to the base.

Generated Plan:

1. (navigate wheeled_rover base loc_1 flat)
2. (collect-sample wheeled_rover sample_a loc_1)
3. (navigate wheeled_rover loc_1 base flat)
4. (deliver-sample wheeled_rover sample_a base)

Analysis: The ENHSP planner successfully generates a minimal, optimal plan. The wheeled rover leverages its capability to traverse flat terrain, navigates to the target, executes the collect-sample action, and returns to the base to satisfy the delivered goal.

2.3 Problem 2: Single Robot, Multiple Samples

Description: An extension of the first problem, requiring the wheeled rover to retrieve two distinct samples scattered across an expanded flat terrain network.

Generated Plan:

1. (navigate wheeled_rover base loc_1 flat)
2. (collect-sample wheeled_rover sample_a loc_1)
3. (navigate wheeled_rover loc_1 base flat)
4. (deliver-sample wheeled_rover sample_a base)
5. (navigate wheeled_rover base loc_1 flat)
6. (navigate wheeled_rover loc_1 loc_2 flat)
7. (collect-sample wheeled_rover sample_b loc_2)
8. (navigate wheeled_rover loc_2 loc_1 flat)
9. (navigate wheeled_rover loc_1 base flat)
10. (deliver-sample wheeled_rover sample_b base)

Analysis: The output demonstrates the planner's ability to handle repetitive task allocation for a single agent. The rover inherently prioritizes sequential execution, completing the entire collection and delivery cycle for sample_a before initiating the deeper traversal required to retrieve sample_b.

2.4 Problem 3: Forced Cooperation via Strict Terrain Constraints

Description: This scenario strictly enforces multi-agent coordination. A sample is located in a rough terrain zone inaccessible to the wheeled_rover. Conversely, the legged_rover cannot traverse the flat terrain leading to the base.

Generated Plan:

1. (navigate legged_rover transfer_zone rocky_crater rough)
2. (collect-sample legged_rover sample_alpha rocky_crater)
3. (navigate legged_rover rocky_crater transfer_zone mixed)

4. (drop-sample legged_rover sample_alpha transfer_zone)
5. (navigate wheeled_rover base transfer_zone mixed)
6. (collect-sample wheeled_rover sample_alpha transfer_zone)
7. (navigate wheeled_rover transfer_zone base flat)
8. (deliver-sample wheeled_rover sample_alpha base)

Coordination Explanation: The planner elegantly solves the capability mismatch by utilizing the transfer_zone (designated as mixed terrain, accessible to both) as a rendezvous point. The coordination occurs in a "relay" fashion:

1. **The Gatherer:** The legged_rover navigates the rough terrain to extract the sample and utilizes the drop-sample action at the transfer zone.
2. **The Courier:** The wheeled_rover arrives at the transfer zone, uses collect-sample to retrieve the payload, and navigates the flat terrain back to the base. This proves that the domain design successfully forces cooperation to achieve the global goal.

2.5 Problem 4: Advanced Coordination with Complex Topology

Description: A highly complex scenario involving a branched crater network. Two samples are located in deep, disconnected rough terrain zones.

Generated Plan:

1. (navigate legged_rover hub crater_south rough)
2. (collect-sample legged_rover sample_s crater_south)
3. (navigate legged_rover crater_south hub mixed)
4. (drop-sample legged_rover sample_s hub)
5. (navigate legged_rover hub crater_north rough)
6. (collect-sample legged_rover sample_n crater_north)
7. (navigate legged_rover crater_north hub mixed)
8. (drop-sample legged_rover sample_n hub)
9. (navigate legged_rover hub crater_south rough)
10. (navigate wheeled_rover base hub mixed)
11. (collect-sample wheeled_rover sample_n hub)
12. (navigate wheeled_rover hub base flat)
13. (deliver-sample wheeled_rover sample_n base)
14. (navigate wheeled_rover base hub mixed)
15. (collect-sample wheeled_rover sample_s hub)
16. (navigate wheeled_rover hub base flat)
17. (deliver-sample wheeled_rover sample_s base)

Coordination Explanation: This represents an advanced asynchronous workflow. The legged_rover acts as a dedicated miner, diving into both the south and north craters consecutively to extract both samples and deposit them at the central hub. Once the mining phase is complete (and the legged rover moves out of the way in step 9), the wheeled_rover acts as a dedicated logistics courier, making multiple round trips between the hub and the base to deliver the samples. The planner optimally distributes the workload based entirely on the mechanical constraints defined in the initial state.

3. Part 2: PDDL+ Hybrid Dynamical System

3.1 Domain Architecture and Hybrid Modeling Strategy

The PDDL+ extension transitions the problem from discrete planning to a Hybrid Dynamical System. This is achieved by introducing continuous time, numeric fluents, processes, and events, strictly adhering to the architectural principles of hybrid planning:

- **Processes (Continuous Flows):** Navigation is modeled as a continuous reduction of distance (navigate-process) based on the rover's speed. Traversing rough terrain acts as a continuous hazard (accumulate-risk), accumulating risk linearly over time. A proactive recovery process (recover-risk) allows rovers to safely deplete risk while resting.
- **Events (Autonomous Limits):** Events define limits and automatically trigger state transitions. The risk-failure event acts as a hard environmental constraint. If risk-level exceeds max-risk, the event automatically triggers, rendering the rover is-broken and dropping its speed to zero.
- **Heuristic Search Mitigation:** To prevent state-space explosion during the ENHSP heuristic search, all continuous processes are strictly conditioned on (not (is-broken ?r)). This guarantees that upon a failure event, all continuous dynamics for the affected rover halt cleanly, allowing the planner to accurately prune the branch without infinite temporal simulation.

3.1.1 State Variables & Numeric Fluents (The Physics Engine)

This section defines both the classical boolean flags (Predicates) and the continuous physical quantities (Functions) required for PDDL+.

Element Type	Name & Parameters	Description & Purpose
Predicate	(path-terrain ?l1 ?l2 ?t)	Crucial Fix: Binds terrain type directly to the <i>edge</i> (path) rather than the node, ensuring risk is only calculated during active traversal.
Predicate	(is-navigating ?r) / (is-resting ?r)	Boolean state flags. Used to lock the rover's state and prevent executing conflicting discrete actions while continuous processes are running.
Predicate	(is-broken ?r)	System failure flag. When true, it permanently halts all continuous processes and discrete actions for that specific rover.
Function	(nav-progress ?r)	Numeric tracker for the remaining distance a rover must cover to reach its target node.
Function	(risk-level ?r)	The core hazard metric. Accumulates dynamically over time in rough terrains and depletes during rest.
Function	(max-risk) / (speed) / (risk-rate)	Static numeric constants defining hardware limitations (maximum tolerated risk, velocity) and environmental hazards.

3.1.2 Discrete Actions (State Transitions & Interventions)

Instead of using durative-actions (which ENHSP does not support well with processes), navigation is decoupled into instantaneous Start/End interventions.

Action Name	Key Preconditions	Key Effects	Purpose & Discussion Context
start-navigate	- (path-terrain) matches (can-traverse) - not (is-broken) - not (is-navigating)	- Flags (is-navigating) - Assigns (nav-progress) = (distance)	Initiates movement. It logically disconnects the rover from its current node and "pushes" it into the continuous edge.
end-navigate	- (is-navigating) is True - (nav-progress) <= 0.0	- Clears (is-navigating) - Updates (at ?to)	Concludes movement. The planner can only execute this once the continuous process has driven the distance down to zero.
start-rest	- Target is not (is-rough) - (risk-level) > 0.0 - not (is-broken)	- Flags (is-resting)	Proactive Safety: Forces the planner to actively seek safe zones to halt and trigger the recovery process before a failure occurs.
stop-rest	- (is-resting) is True	- Clears (is-resting)	Ends the recovery phase so the rover can resume its mission.
Manipulation Actions	- not (is-navigating) - not (is-resting) - not (is-broken)	- Updates (carrying) / (empty)	Standard pickup/drop/deliver actions, now heavily guarded by safety and state-lock predicates to ensure physical realism.

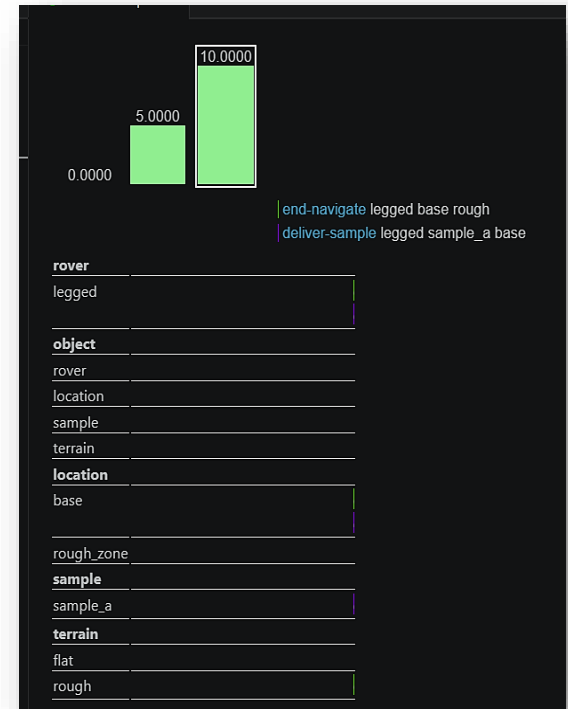
3.1.3 Continuous Dynamics (Processes & Events)

This is the heart of PDDL+. It dictates how the environment changes automatically over time (#t) without explicit planner actions.

Dynamic Type	Name	Trigger Conditions (Preconditions)	Continuous / Autonomous Effects	Purpose & Discussion Context
Process	Maps-process	- (is-navigating) is True - not (is-broken)	decrease (nav-progress) by (speed * #t)	Simulates continuous physical motion. Progress reduces linearly over time based on the rover's specific velocity.
Process	accumulate-risk	- (is-navigating) is True - Path (is-rough) - not (is-broken)	increase (risk-level) by (risk-rate * #t)	The Hazard Engine: Automatically penalizes the rover for spending time in rough terrain.
Process	recover-risk	- (is-resting) is True - not (is-broken)	decrease (risk-level) by (15.0 * #t)	Represents cooling down or repairing. Risk drops rapidly over time as long as the rover remains in a resting state.
Event	risk-failure	- (risk-level) > (max-risk) - not (is-broken)	- Sets (is-broken) to True - Sets (speed) to 0.0	The Autonomous Constraint: Acts as a physics-based hard limit. If the planner ignores the risk, this event fires automatically, killing the rover and pruning that invalid trajectory from the search space.

3.2 Problem 1: Baseline Continuous Dynamics

- **Objective:** Validate the fundamental continuous processes (distance depletion and risk accumulation) in a simple single-agent scenario.
- **Execution Analysis:** The ENHSP planner synthesized an optimal 16-step plan with an elapsed time of 10.0 units. The legged rover starts navigating to the rough_zone. The planner correctly outputs -----waiting----- [5.0], indicating the delegation of state updates to the Maps-process. Since the distance is 10.0 and speed is 2.0, the process takes precisely 5.0 time units. The rover collects the sample and returns.
- **Conclusion:** The baseline continuous physics engine operates flawlessly. Risk accumulation remains under the critical threshold, requiring no proactive resting interventions.



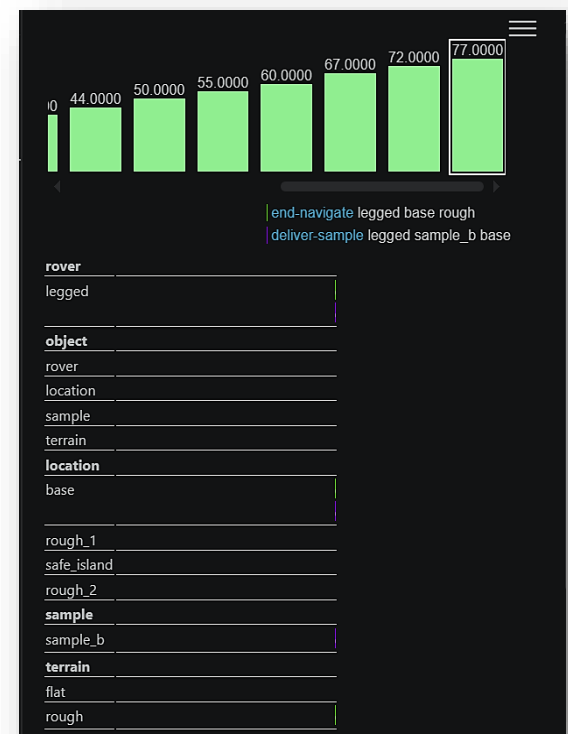
3.3 Problem 2: Proactive Risk Mitigation via Mandatory Resting

- **Objective:** Force the planner to autonomously reason about numeric risk thresholds and utilize the safe_island intermediate node to avoid system failure.

- **Execution Analysis:**

The generated plan spans 107 steps over 77.0 time units. The mathematical constraints strictly dictated this behavior:

- ❖ Each topological leg (distance 9.0 at speed 2.0) requires 4.5 time units. With a risk-rate of 10.0, each leg generates 45.0 risk units.
- ❖ A direct round trip without resting would yield 90.0 risk (Safe). However, proceeding sequentially to the target and back would exceed the max-risk of 100.0, triggering the risk failure event. The planner successfully identified this impending hazard.
- ❖ The output explicitly shows the rover navigating to safe_island, executing (start-rest legged safe_island flat), and initiating a waiting period to allow the recover-risk process to deplete the accumulated risk before proceeding to rough_2.
- **Conclusion:** The planner successfully synthesized a safety-critical trajectory, proving the robustness of the risk-management modeling.



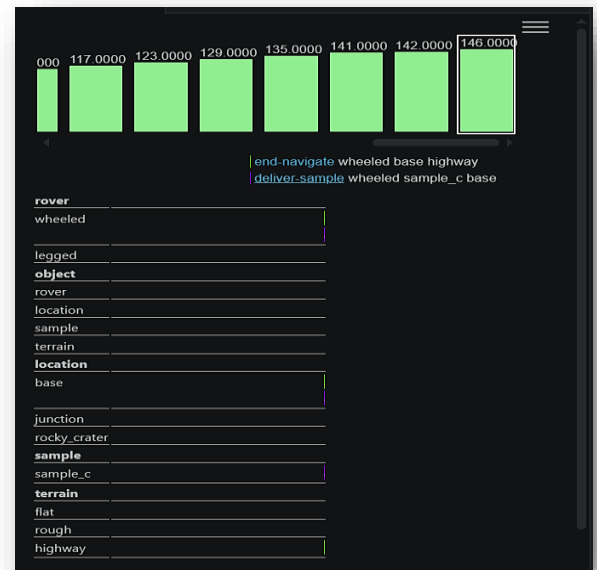
3.4 Problem 3: Time-Critical Heterogeneous Cooperation

- **Objective:** Demand asynchronous, multi-agent cooperation enforced by spatial and terrain constraints (the introduction of the highway terrain, accessible only to the wheeled rover).
- **Execution Analysis:**

The planner generated a highly complex 202-step plan with an elapsed time of 146.0.

- ❖ **The Legged Rover (Gatherer):** Dispatched into the hazardous rough terrain to retrieve sample_c. Due to the state-space complexity and heuristic search dynamics, the planner utilizes heavy oscillation (repetitive start-rest and short navigational bounds) to carefully manage the risk gradient while waiting for the wheeled rover's synchronization. It drops the sample at the junction.
- ❖ **The Wheeled Rover (Courier):** At timestamp 67.0, the wheeled rover utilizes the highway to reach the junction, collects the sample at 71.0, and returns to the base for the final delivery at 146.0.

- **Conclusion:** The scenario successfully forces inter-agent dependency. The legged rover handles the physical manipulation in hazardous zones, while the wheeled rover exploits its speed advantage on the highway for logistics. The extended plan length is a direct manifestation of the heuristic search navigating an infinite continuous state-space to find a feasible temporal synchronization.



3.5 Problem 4: Scaled Multi-Agent Optimization

- **Objective:** A comprehensive benchmark utilizing the (:metric minimize (total-time)) constraint, requiring the concurrent scheduling of two rovers handling multiple targets across complex topology.

- **Execution Analysis:**

The planner synthesized a highly optimized 152-step parallel plan with an elapsed total time of 108.0.

- ❖ **Concurrency:** At timestamp 0.0, the legged rover begins navigating to the north_crater. Simultaneously, at timestamp 5.0, the wheeled rover begins utilizing the highway to reach the transfer_point.
- ❖ **Task Allocation & Handover 1:** The legged rover extracts sample_n and deposits it at the transfer_point at timestamp 16.0. The wheeled rover immediately collects it and returns to base, delivering it at 21.0.



- ❖ **Task Allocation & Handover 2:** While the wheeled rover is delivering the first sample, the legged rover does not idle; it immediately begins navigating to the south_crater at 21.0 to extract sample_s. It drops it at the transfer point at 37.0 and rests. The wheeled rover returns via the highway at 98.0, collects the second sample at 103.0, and finalizes the mission at 108.0.
- **Conclusion:** This output represents the pinnacle of hybrid planning efficiency. The planner achieved maximum parallelization, minimizing idle times and optimally matching hardware capabilities (speed vs. terrain resilience) to task requirements, entirely avoiding system failure events.

4. Final Results and Limitations

4.1 Temporal and State-Space Complexity

Introducing heterogeneous agents significantly impacts planning complexity. While hardware constraints dynamically prune invalid topological branches, the temporal complexity explodes in PDDL+. Orchestrating asynchronous, parallel tasks across agents with varying speeds and risk tolerances causes combinatorial state-space expansion. By explicitly bounding continuous processes (halting them upon failure events), the domain successfully guides the ENHSP heuristic, avoiding infinite temporal dead-ends.

4.2 Limitations of Symbolic Modelling (The TMP Gap)

While PDDL+ excels at high-level reasoning, it abstracts terrain into discrete boolean states (e.g., (is-rough)) and uniform numerical risk rates. This creates a fundamental gap in **Task-Motion Planning (TMP)**. It abstracts away vital physical realities such as slope gradients, traction coefficients, and precise kinodynamic constraints. The task planner assumes unconditional geometric reachability within a node; thus, a symbolically valid plan might fail during low-level control execution if the continuous physical environment presents unmodeled obstacles or varying friction parameters. Future real-world deployment would require integrating the symbolic planner with a geometric motion planner (e.g., using RRT) operating in the belief space to handle sensor noise and execution uncertainties.

5. Conclusion

This project successfully demonstrates the application of AI planning in heterogeneous multi-robot systems, transitioning from classical discrete modeling to Hybrid Dynamical Systems. In Q1, by utilizing a capability-based STRIPS model (**can-traverse**), the ENHSP planner synthesized emergent, forced cooperation between rovers, organically resolving complex spatial constraints without explicit coordination actions. In Q2, the PDDL+ extension incorporated continuous time ($\#t$) to model real-world physical dynamics, specifically continuous navigation, linear risk accumulation, and proactive recovery.

The planner effectively managed concurrent processes and the autonomous **risk-failure** event, dynamically scheduling mandatory resting phases to prevent systemic failure. Ultimately, this work highlights the inherent trade-off between spatial pruning through heterogeneity and the severe temporal state-space explosion characteristic of hybrid planning, while exposing the fundamental Task-Motion Planning (TMP) gap when abstracting physical terrain dynamics into symbolic states.